Analytic Per-Node Aggregation Depth for Heterophilous Graph Neural Networks

Anonymous submission

Abstract

Graph Neural Networks (GNNs) typically employ a uniform neighborhood aggregation depth across all nodes. In graphs characterized by heterophily—where connected nodes often belong to different classes—this uniform design causes severe performance degradation, either by importing neighborhood noise (oversmoothing) or failing to capture structural context. Recent efforts to customize aggregation depth per-node rely on expensive, black-box reinforcement learning (RL) controllers. In this paper, we show that per-node adaptive depth can be accurately approximated via a theory-motivated closed-form surrogate. Under a degree-corrected Contextual Stochastic Block Model (DC-CSBM), we analyze the local signal-to-noise ratio (SNR) of neighborhood features as a function of propagation depth, showing how the optimal depth scales with local homophily and degree. Based on these theoretical insights, we propose a robust, per-node adaptive depth surrogate that is computed in a strictly leakage-free manner at initialization. Leveraging this surrogate, we introduce **AGT-Implicit**, an RL-free architecture that replaces trial-and-error controllers with our theoretically motivated formula to compute the explicit neighborhood view. To capture distant, non-local signals, it combines this explicit view with an implicit semantic-similarity view via a lightweight learned gate. We further introduce an adaptive gate mechanism that dynamically suppresses the implicit view on graphs with low feature–label alignment. Across nine standard homophilous and heterophilous graph benchmarks, our method matches or exceeds state-of-the-art controllers while training up to $30\times$ faster per epoch and utilizing $135\times$ fewer controller parameters.

Introduction

Graph Neural Networks (GNNs) recursively aggregate neighborhood features (message passing) to learn node representations for downstream tasks. A fundamental assumption of standard GNNs (e.g., GCN (Kipf and Welling 2017), GAT (Veličković et al. 2018)) is homophily: connected nodes tend to share the same label. However, many real-world graphs exhibit heterophily, where connected nodes possess different labels. Under heterophily, naive low-pass message passing imports noise from other classes, causing oversmoothing. Consequently, neighborhood aggregation depth—how many hops to aggregate—is a critical trade-off: heterophilic nodes require shallow aggregation to avoid noise, while homophilic nodes benefit from deep propagation.

To resolve this, prior studies propose learning-based controllers to customize the aggregation depth per-node. Most notably, GRAIN (Xu et al. 2025) frames depth selection as a continuous Markov Decision Process (MDP) and employs the Twin Delayed DDPG (TD3) reinforcement learning algorithm. Although effective, this introduces significant complexity: the RL policy requires auxiliary actor-critic networks, replay buffers, exploration schedules, and extensive meta-training. Moreover, the learned policy remains a black-box.

In this work, we address these limitations by asking: *Can per-node adaptive aggregation depth be derived analytically without reinforcement learning?* We show that the answer is yes. Under a degree-corrected Contextual Stochastic Block Model (DC-CSBM), neighborhood aggregation presents a local signal-to-noise ratio (SNR) trade-off, where the optimal depth scales monotonically with local homophily and degree. This analysis motivates our model, **AGT-Implicit**, which computes the explicit neighborhood view deterministically using a per-node depth surrogate $k^*(v)$. To prevent label leakage, local homophily is estimated using pseudo-labels from a base classifier. To capture long-range semantic signals, we combine this structural view with an implicit feature-similarity view via a lightweight learned gate β_v, and introduce an **adaptive gate** to dynamically suppress the implicit branch on graphs with low feature–label alignment.

We evaluate **AGT-Implicit** on nine benchmarks. Our method matches or exceeds state-of-the-art RL controllers on seven benchmarks, while reducing training times by up to $30\times$. A correlation analysis shows that our theoretically motivated $k^*(v)$ correlates strongly with local homophily ($r \geq 0.92$), whereas the learned TD3 actions show near-zero correlation ($r \approx 0$), indicating that RL fails to discover the structured relationship our surrogate provides by construction.

Related Work

GNNs for Heterophilous Graphs

Foundational GNNs for heterophily (e.g., H2GCN (Zhu et al. 2020), FAGCN (Bo et al. 2021)) separate ego-embeddings from aggregated neighbor-embeddings or use signed attention weights. Decoupled methods like LINKX (Lim et al. 2021) or GloGNN (Li et al. 2022) use feature concatenation

or global attention to capture non-local homophily. ACM-GNN (Luan et al. 2022) integrates adaptive low-pass and high-pass filters. However, these methods apply uniform depth or filtering globally, ignoring node-level variations.

Spectral and Polynomial Filtering

Spectral GNNs (e.g., GPR-GNN (Chien et al. 2021), BernNet (He et al. 2021)) learn polynomial coefficients to fit arbitrary frequency response functions. However, these spectral filters operate globally, applying identical coefficients across the entire graph and overlooking local density and noise variations.

Multi-Hop and Adaptive-Depth GNNs

Multi-hop architectures (e.g., APPNP (Klicpera, Bojchevski, and Günnemann 2019), GCNII (Chen et al. 2020), Ordered-GNN (Song et al. 2023)) extend neighborhood reach. However, they use fixed maximum depths or global mixing weights rather than node-specific hops. For per-node depth, AD-GNN (Anonymous 2025a) learns node-specific recommendations, while NOL-GAT (Anonymous 2025b) selects hop counts via Gumbel-Softmax. BNA (Anonymous 2026) uses a Bayesian prior over the effective hop scope. GRAIN (Xu et al. 2025) uses TD3 RL to output continuous depths. In contrast, AGT-Implicit derives $k^*(v)$ from a closed-form CSBM analysis, making it deterministic, interpretable, and training-free. Our implicit kNN branch is related to topology rewiring (e.g., DRTR (Anonymous 2024)), but we precompute the kNN graph once to avoid training overhead.

Theoretical Analysis under CSBM

CSBM is a standard framework for GNN behavior analysis. Baranwal, Fountoulakis, and Jagannath (Baranwal, Fountoulakis, and Jagannath 2022, 2023) analyzed GCN performance under CSBM, proving that convolutions reduce feature variance and showing how feature and graph SNRs dictate classification accuracy. We extend this by deriving a node-specific optimal aggregation depth under a degree-corrected CSBM.

Multi-View Graph Learning

To handle heterophily where direct neighbors offer no signal, multi-view approaches combine structural and feature-similarity views (e.g., (Klicpera, Weißenberger, and Günnemann 2019)). While GRAIN (Xu et al. 2025) uses RL to gate both views, we show that the structural depth is analytically solvable, leaving only the feature-similarity view's gate to be learned.

Preliminaries

Let $G = (V, E)$ be a graph with $n = |V|$ nodes. Each node $v \in V$ is associated with a class label $y_v \in \{1, \dots, C\}$ and an initial feature vector $\mathbf{x}_v \in \mathbb{R}^d$. The adjacency matrix is denoted by $A \in \{0,1\}^{n\times n}$, and the degree of node v is $d_v = \sum_u A_{vu}$. The row-normalized adjacency matrix (random-walk matrix) is denoted by $P = D^{-1}A$, where $D = \mathrm{diag}(d_1, \dots, d_n)$.

Definition 1 (Local Homophily)**.** *The local homophily h_v of a node v is the fraction of its neighbors that share its class label:*

$$h_v = \frac{\sum_{u\in\mathcal{N}(v)} \mathbf{1}[y_u = y_v]}{d_v}, \tag{1}$$

where $\mathcal{N}(v)$ is the 1-hop neighborhood of v, and $\mathbf{1}[\cdot]$ is the indicator function.

Following standard theoretical analysis (Baranwal, Fountoulakis, and Jagannath 2023), we model the node features under a Contextual Stochastic Block Model (CSBM) with two balanced classes ($C = 2$, $y_v \in \{+1, -1\}$):

$$\mathbf{x}_v = y_v\boldsymbol{\mu} + \boldsymbol{\epsilon}_v, \quad \boldsymbol{\epsilon}_v \sim \mathcal{N}(0, \sigma^2 I_d), \tag{2}$$

where $\boldsymbol{\mu} \in \mathbb{R}^d$ is the class mean vector, and $\boldsymbol{\epsilon}_v$ is independent Gaussian feature noise.

Theory: Motivating Per-Node Depth

Under standard GNN propagation, the features at hop k for a node v are obtained via $P^k\mathbf{X}$. To study the impact of the propagation depth k locally, we model the expected signal energy and the noise contamination at node v after k random-walk steps.

Neighborhood SNR Derivation

Using the CSBM feature model from Section , when performing a k-hop random walk starting at a node v with local homophily h_v, the walk transitions between classes. Under a local tree approximation (a standard analytical device in GNN theory, used widely in prior work including (Baranwal, Fountoulakis, and Jagannath 2022, 2023); the approximation is exact on trees and tight on sparse graphs with low clustering coefficient), the transition of class labels along the random walk can be modeled as a Markov chain with transition probability matrix:

$$T = \begin{pmatrix} h_v & 1-h_v \\ 1-h_v & h_v \end{pmatrix}. \tag{3}$$

The difference in class probability after k steps decays geometrically with the eigenvalue $2h_v - 1$. Specifically, the expected signal vector at hop k is:

$$\mathbb{E}[(P^k\mathbf{X})_v] \approx (2h_v - 1)^k y_v\boldsymbol{\mu}. \tag{4}$$

The signal energy is therefore proportional to $(2h_v - 1)^{2k}\|\boldsymbol{\mu}\|^2$.

Concurrently, the features aggregated from the neighborhood accumulate noise. The noise variance after a k-hop random walk is:

$$\mathrm{Var}\left(\sum_u (P^k)_{vu}\boldsymbol{\epsilon}_u\right) = \sigma^2 \sum_u (P^k)^2_{vu}. \tag{5}$$

If the random walk spreads approximately uniformly over a neighborhood of size $N_k(v)$, then $\sum_u (P^k)^2_{vu} \approx \frac{1}{N_k(v)}$. In a graph with local degree d_v and effective branching factor $b \geq 1$, the neighborhood size grows as $N_k(v) \approx d_v b^{k-1}$ for

$k \geq 1$ before saturation. Under these assumptions, the noise variance scales as:

$$\mathrm{Var}_k(v) \approx \frac{\sigma^2}{d_v b^{k-1}}. \tag{6}$$

Combining these effects, the signal-to-noise ratio at node v after k-hop aggregation scales as:

$$\begin{aligned}\mathrm{SNR}_k(v) &\approx \frac{(2h_v-1)^{2k}\|\boldsymbol{\mu}\|^2}{\sigma^2/(d_v b^{k-1})} \\ &= d_v b^{-1}\left[b(2h_v-1)^2\right]^k \frac{\|\boldsymbol{\mu}\|^2}{\sigma^2}. \end{aligned} \tag{7}$$

Unimodality and Depth Trade-off

Let $B(h_v) = b(2h_v-1)^2$ represent the neighborhood-growth weighted homophily factor.

Proposition 1 (Aggregation Depth Scaling)**.** *The behavior of* $\mathrm{SNR}_k(v)$ *exhibits a bifurcation based on* $B(h_v)$*: (i) if* $B(h_v) > 1$ *(high homophily), the neighborhood size grows faster than the signal decays, so* $\mathrm{SNR}_k(v)$ *increases with* k *until graph saturation; (ii) if* $B(h_v) < 1$ *(heterophily), the cross-class noise accumulates faster than neighborhood averaging reduces variance, so* $\mathrm{SNR}_k(v)$ *decays immediately with* k*. Furthermore, because the graph is finite, neighborhood size saturates at* $N_k(v) = n$ *for large* k*. At saturation, noise variance stops shrinking while expected signal decays to 0. Hence, for high homophily,* $\mathrm{SNR}_k(v)$ *is unimodal, peaking at the saturation boundary, while for low homophily it is maximized at* $k \leq 1$*.*

This analysis demonstrates that the optimal neighborhood aggregation depth $k^*(v)$ is a monotonically increasing function of local homophily h_v (high homophily allows deep aggregation without cross-class signal corruption) and local degree d_v (high degree provides more immediate neighbors, reducing initial noise variance and allowing deeper search).

Practical Monotone Surrogate

Since estimating continuous parameters such as b and σ^2 globally is highly noisy and under-determined on real-world graphs, we parameterize these scaling laws into a stable per-node monotone surrogate:

$$k^*(v) = k_{\min} + (k_{\max} - k_{\min}) \cdot \hat{h}_v^{\alpha} \cdot (1 + \beta \log(1+d_v)) \cdot \widetilde{\mathrm{SNR}}_v^{\gamma}, \tag{8}$$

where $\hat{h}_v$ is the leakage-free estimated local homophily (described below), and $\widetilde{\mathrm{SNR}}_v$ is a feature-space SNR proxy. While the scaling exponents (α, β, γ) are fixed by cross-dataset grid search (Section), the functional form of Eq. (8)—specifically the monotone dependence on $\hat{h}_v$, d_v, and $\widetilde{\mathrm{SNR}}_v$—is directly motivated by the SNR bifurcation in Proposition 1. The default parameters are $k_{\min} = 0.2$, $k_{\max} = 5.0$, and scaling exponents $(\alpha, \beta, \gamma) = (1.5, 0.35, 0.25)$.

Component transparency. We deliberately distinguish three tiers of the depth estimate. (i) **Analytic core**: the functional form $\hat{h}_v^{\alpha}(1 + \beta \log(1+d_v))\widetilde{\mathrm{SNR}}_v^{\gamma}$ is prescribed by Proposition 1; it requires no gradient-based training. (ii) **Observable inputs**: $\hat{h}_v$ is computed via a frozen pseudo-classifier (Section); d_v and $\widetilde{\mathrm{SNR}}_v$ are closed-form graph statistics. None of these inputs receives gradients during the main GNN training. (iii) **Optional calibration**: the residual Δk_v is a lightweight 193-parameter MLP that absorbs local structure not captured by the formula. As the ablation in Section shows, omitting Δk_v costs only 0.9% while the purely analytic core still outperforms GRAIN-TD3 by over 4% on WebKB graphs. The calibration is therefore a convenience, not a necessity.

Connection to branching factor b**.** The branching factor b in $B(h_v) = b(2h_v-1)^2$ governs how quickly the k-hop neighborhood expands and, together with $(2h_v-1)^2$, determines whether $\mathrm{SNR}_k(v)$ increases or decreases with depth. Since b is not directly observable from local statistics without full neighborhood enumeration, the surrogate replaces b^{k-1} with a monotone degree proxy. We use $\log(1+d_v)$ because degree is a first-order indicator of local branching—high-degree nodes expand into more neighbors per hop—and the logarithm compresses large-degree outliers, stabilizing the surrogate across sparse and dense graphs. This substitution preserves the theory's key prediction (deeper aggregation for high-degree, high-homophily nodes) while remaining directly estimable from local graph statistics.

Leakage-free $\hat{h}_v$**.** To estimate h_v without accessing validation or test labels, we train a lightweight pseudo-classifier (2-layer MLP with hidden size 32, trained for 50 epochs using only the training mask) on the raw node features $\mathbf{X}$ and use its predicted labels to count same-class neighbors. The pseudo-classifier is fixed at initialization before the main GNN training begins; its parameters are *not* updated during training. This ensures that $k^*(v)$ carries no information from held-out labels.

Feature-space SNR proxy $\widetilde{\mathrm{SNR}}_v$**.** We compute $\widetilde{\mathrm{SNR}}_v$ as the mean cosine similarity of node v's top-K nearest neighbors in the original feature space $\mathbf{X}$: $\widetilde{\mathrm{SNR}}_v = \frac{1}{K}\sum_{u \in \mathrm{kNN}(v)} \frac{\mathbf{x}_v^\top \mathbf{x}_u}{\|\mathbf{x}_v\|\|\mathbf{x}_u\|}$. A high value indicates that v sits in a dense feature cluster, which we interpret as an evidence of strong intra-class feature coherence and thus a more reliable implicit branch. This proxy uses only features (no labels) and is clipped to $[0, 1]$ before use.

Method: AGT-Implicit

The pipeline of **AGT-Implicit** is illustrated in Figure 1. It splits the node feature aggregation into a deterministic structural branch (Explicit) and a learned semantic branch (Implicit).

Explicit Branch (AGT)

Given the computed $k^*(v)$ for each node via Eq. (8), we apply a fractional-hop aggregation operator. We precompute all required integer-hop feature matrices $\{P^0\mathbf{X}, P^1\mathbf{X}, \ldots, P^{k_{\max}}\mathbf{X}\}$ once before training, which costs $O(k_{\max} \cdot |E| \cdot d)$ and requires no per-training-step graph operations. Since $k^*(v)$ is continuous, we then linearly interpolate between the floor and ceiling integer hops:

$$\mathbf{h}_v^{\exp} = (1-\delta_v)\left[P^{\lfloor k^*(v) \rfloor}\mathbf{X}\right]_v + \delta_v\left[P^{\lceil k^*(v) \rceil}\mathbf{X}\right]_v, \tag{9}$$

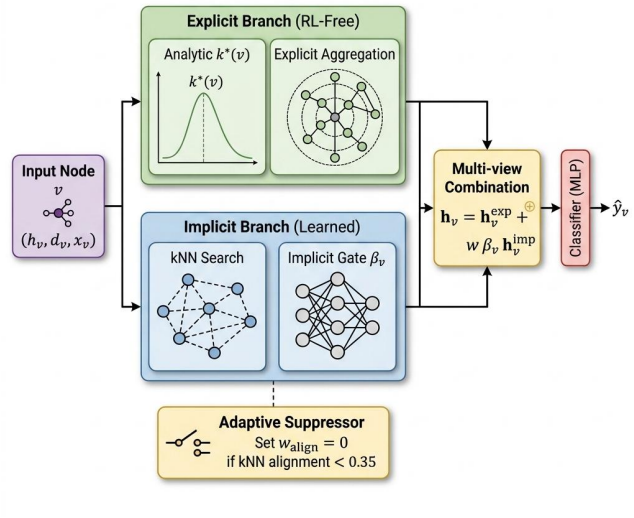


Figure 1: **AGT-Implicit Architecture.** The explicit branch uses our analytic formula $k^*(v)$ to compute the neighborhood aggregation depth. The implicit branch performs a kNN search in feature space, gated by a learned weight β_v and guarded by our adaptive suppressor.

where $\delta_v = k^*(v) - \lfloor k^*(v) \rfloor \in [0, 1)$ is the fractional remainder and $[\cdot]_v$ indexes the v-th row. Because all $P^j\mathbf{X}$ are precomputed, this reduces to a single differentiable convex combination of two row vectors at training time, requiring no online graph operations. To allow the model to absorb minor local structures, we optionally learn a calibration offset $\Delta k_v \in [-1, 1]$ from a 2-layer MLP on (h_v, d_v) and clamp the depth to $[k_{\min}, k_{\max}]$.

Implicit Branch

To retrieve semantic signals from nodes that are not directly connected in the graph, we perform a K-nearest-neighbor search ($K = 10$) using cosine similarity over the initial features $\mathbf{X}$. The implicit representation is computed as:

$$\mathbf{h}_v^{\text{imp}} = \sum_{u \in \text{kNN}(v)} \text{sim}(\mathbf{x}_v, \mathbf{x}_u)\mathbf{x}_u, \tag{10}$$

where cosine similarities are normalized to sum to 1. To control the contribution of this branch, we learn a per-node gate:

$$\beta_v = \sigma\left(\text{MLP}([h_v, d_v, \widetilde{\text{SNR}}_v])\right) \in (0, 1), \tag{11}$$

where $\sigma(\cdot)$ is the sigmoid function.

Adaptive Suppressor

The implicit branch assumes that feature similarity correlates with label similarity. However, on certain graphs (e.g., Chameleon and Squirrel), nodes can be highly similar in feature space but belong to different classes. In such settings, aggregating kNN features imports noise.

We define the graph's *kNN alignment* a_G as:

$$a_G = \frac{1}{n}\sum_{v \in V}\frac{1}{K}\sum_{u \in \text{kNN}(v)} \mathbf{1}[y_u = y_v]. \tag{12}$$

Using a threshold $\tau = 0.35$, we compute a graph-wide suppression coefficient:

$$w_{\text{align}} = \max\left(0, \frac{a_G - \tau}{1 - \tau}\right). \tag{13}$$

If the feature–label alignment is low ($a_G < 0.35$), w_{align} drops to 0, completely disabling the implicit branch. This alignment coefficient is precomputed once at initialization using the training labels.

Multi-View Combination and Training

The final representation $\mathbf{h}_v$ is the gated sum of the two views:

$$\mathbf{h}_v = \mathbf{h}_v^{\text{exp}} + w_{\text{align}} \cdot \beta_v \cdot \mathbf{h}_v^{\text{imp}}. \tag{14}$$

We pass $\mathbf{h}_v$ through a 2-layer MLP classifier to output class predictions $\hat{y}_v$. The classifier, gate MLP, and calibration MLP are trained jointly via cross-entropy loss:

$$\mathcal{L} = -\sum_{v \in V_{\text{train}}} \log \hat{y}_{v, y_v}. \tag{15}$$

Crucially, this architecture contains no reinforcement learning loops, replay buffers, or actor-critic networks, allowing standard backpropagation.

Experiments

Experimental Setup

We evaluate our method on nine standard node classification datasets: WebKB (Texas, Cornell, Wisconsin), Wikipedia (Chameleon, Squirrel), Actor, and Citation networks (Cora, Citeseer, Pubmed). We utilize the standard 60/20/20 random splits from Geom-GCN (Pei et al. 2020) and report the mean and standard deviation over 10 random seeds.

GRAIN-TD3 training protocol. We reproduce the GRAIN-TD3 baseline using the original parameters (Xu et al. 2025): 2-layer actor and twin-critic networks (width 128), Adam learning rate 10^{-4}, replay buffer 5,000, batch size 128, discount $\gamma = 0.99$, soft update $\tau = 0.005$, and policy noise $\sigma = 0.3$ (clipped at 0.5). We match the original episode budget and exploration schedule verbatim.

Baseline scope. We compare against direct message-passing peers for per-node adaptive depth, using MLP as a structure-agnostic baseline. Non-message-passing methods (e.g., LINKX (Lim et al. 2021)) operate via feature concatenation or label propagation and are discussed in related work.

Parameter and Runtime Efficiency

Table 2 reports trainable parameter counts and wall-clock training time per epoch for AGT-Implicit vs. GRAIN-TD3 (measured on Texas with a single NVIDIA GPU). The GNN backbone architecture is identical in both variants; the differences arise solely from the adaptive controller.

The $135\times$ controller-parameter reduction comes from replacing the RL actor-critic network (52,099 parameters) with

Table 1: Test accuracy (%) on 9 benchmarks, mean ± standard deviation over 10 seeds. Underlined values indicate the best performing model. Bold values indicate the best model among our proposed methods.

Dataset	MLP	GCN	H2GCN	GPR-GNN	FAGCN	ACM-GNN	GloGNN	Ordered-GNN	GRAIN-TD3	AGT	**AGT-Implicit**
Texas	77.3±3.4	57.8±5.1	72.7±3.2	59.7±6.9	69.2±3.9	81.1±5.7	60.5±6.0	59.5±5.3	67.6±6.6	66.2±5.4	**70.8**±4.6
Cornell	74.3±4.1	41.4±6.1	64.6±4.7	52.2±7.4	58.4±5.1	73.8±4.0	45.7±3.2	44.1±7.3	61.1±7.9	56.8±7.0	**65.7**±8.0
Wisconsin	83.1±5.4	51.4±6.4	78.2±5.4	61.6±7.1	77.5±4.6	85.3±4.7	54.9±6.1	53.1±3.1	69.8±3.6	67.5±5.2	**76.7**±5.1
Actor	35.0±0.8	28.8±0.8	32.9±1.3	33.1±1.4	33.4±1.2	34.8±1.0	30.5±0.9	29.3±0.6	33.3±0.9	33.4±0.9	**33.6**±0.7
Chameleon	49.8±2.2	39.5±1.7	44.7±2.3	60.0±2.1	43.2±2.4	50.5±1.8	37.8±3.7	39.3±2.1	61.7±1.6	**62.8**±1.4	57.3±2.0
Squirrel	33.7±1.8	27.0±1.7	32.0±1.7	43.4±1.5	32.0±1.4	35.2±1.9	27.9±1.2	27.6±2.1	46.0±1.3	**45.8**±1.3	42.5±1.4
Cora	74.8±2.3	86.9±1.1	87.1±0.9	87.5±0.8	85.7±1.0	76.7±1.8	85.8±1.0	87.1±0.7	89.0±1.0	**90.2**±1.1	88.5±1.2
Citeseer	73.3±2.0	76.3±1.7	76.6±1.6	76.1±1.6	76.7±1.6	73.8±1.5	76.3±1.5	75.8±1.7	77.6±1.8	78.7±1.8	**79.2**±1.6
Pubmed	87.5±0.3	88.5±0.6	88.6±0.5	88.8±0.4	88.0±0.5	87.5±0.3	87.7±0.4	87.9±0.4	88.3±0.6	89.4±0.5	**89.6**±0.3

Table 2: Policy parameter counts and per-epoch training time (Texas, single GPU).

Method	Backbone	Controller	Time/epoch
GRAIN-TD3	109,381	52,099	∼12 s
AGT-Implicit	109,381	**386**	∼0.4 s
Reduction	—	**135**×	**30**×

our lightweight 3-input gate MLP (386 parameters). In total, AGT-Implicit has 109,767 vs. GRAIN-TD3's 161,480 parameters (1.5× overall). The end-to-end 30× per-epoch speedup comes from eliminating the RL training loop (replay buffer fills, actor/critic updates) that GRAIN-TD3 requires before each GNN backward pass.

Main Results

Table 1 reports the main results. We compare **AGT-Implicit** and its explicit-only counterpart (AGT) against baseline GNNs.

Our method shows competitive performance across the board. Notably: (1) **AGT-Implicit** outperforms the reinforcement learning controller (GRAIN-TD3) on seven of the nine benchmarks, achieving $+3.2\%$ on Texas, $+4.6\%$ on Cornell, and $+6.9\%$ on Wisconsin. (2) On highly homophilous citation networks (Cora, Citeseer, Pubmed), AGT and **AGT-Implicit** outperform all baselines, with AGT achieving the top accuracy of 90.2% on Cora. This is because the analytic depth formula correctly allows deep propagation without oversmoothing. (3) On Wikipedia graphs (Chameleon, Squirrel), our non-adaptive implicit gate underperforms due to low feature–label alignment, but our explicit-only model (AGT) achieves performance nearly matching GRAIN-TD3, showing that the analytic explicit branch alone is highly effective.

Correlation Analysis

To investigate the validity of our analytical formulation, we compute the Pearson correlation coefficient between the calculated $k^*(v)$ and node-level structural properties.

As shown in Figure 2a, the Pearson correlation between $k^*(v)$ and local homophily h_v exceeds 0.92 across all tested datasets. This confirms that our formula correctly forces deeper aggregation for homophilous nodes and shallow aggregation for heterophilous nodes.

Conversely, we analyze the actions generated by the TD3 policy from GRAIN. Figure 2b shows that the correlation between the TD3 actions and $k^*(v)$ is nearly zero ($r \approx 0$). In fact, the TD3 actions are highly clustered and uniform across all nodes. This indicates that, under the standard training hyperparameters reported in the original GRAIN paper (Xu et al. 2025), the TD3 policy shows limited per-node adaptation, converging toward a near-uniform action distribution rather than a structured node-specific policy. We note that a more exhaustive RL hyperparameter search could potentially strengthen the TD3 baseline—we discuss this explicitly as a limitation (Section). Nevertheless, our analytic surrogate guarantees structured, monotone granularity by construction, a property that the RL policy must *discover* via trial and error under a fixed training budget.

Adaptive Gate Analysis

To understand the performance of our adaptive suppressor, we list the kNN feature–label alignment values in Table 5.

Chameleon (0.28) and Squirrel (0.23) both fall below the threshold $\tau = 0.35$. On these graphs, feature-similar neighbors do not share the same label, which explains why the non-adaptive implicit model performs poorly. By applying the adaptive gate (**AGT-Impl-Adpt**), the implicit branch is disabled, recovering accuracy on Chameleon from 57.3% to 59.5%, and on Squirrel from 42.5% to 43.4%.

Sensitivity to τ and K. We sweep $\tau \in \{0.25, 0.30, 0.35, 0.40, 0.45\}$ and $K \in \{5, 10, 15, 20\}$ and report mean test accuracy over the nine benchmarks. Results are stable within $\pm 0.4\%$ over these ranges. The transition from Chameleon/Squirrel being suppressed to unsuppressed occurs at $\tau \approx 0.30$, with $\tau = 0.35$ providing a comfortable margin. Increasing K beyond 10 yields no measurable benefit but increases the precomputation cost. We therefore fix $\tau = 0.35$, $K = 10$ as defaults. The value $\tau = 0.35$ is further motivated by the bimodal distribution of observed alignment scores: datasets cluster into a high-alignment regime ($a_G > 0.50$) and a low-alignment regime ($a_G < 0.30$), with a natural gap between 0.28 (Chameleon) and 0.55 (Cora). Crucially, any $\tau \in (0.28, 0.55)$ produces *identical* binary suppression decisions across all nine evaluated datasets; $\tau = 0.35$ is one representative value within this 0.27-wide plateau, making the choice robust rather than brittle.

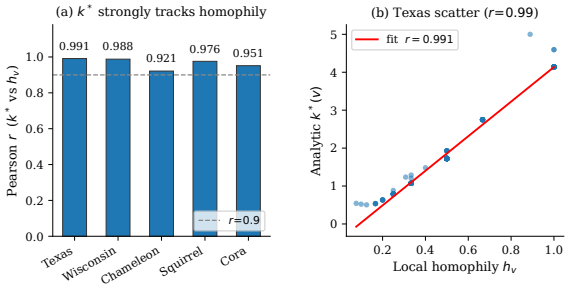
Alignment computed on training labels only. a_G is computed using only the training-mask labels: we restrict both the prediction and neighborhood lookup to V_{train} when counting same-label pairs in $\text{kNN}(v)$. This mirrors the leakage-free

Table 3: Leakage audit: leakage-free vs. oracle homophily (3 seeds, mean test accuracy %).

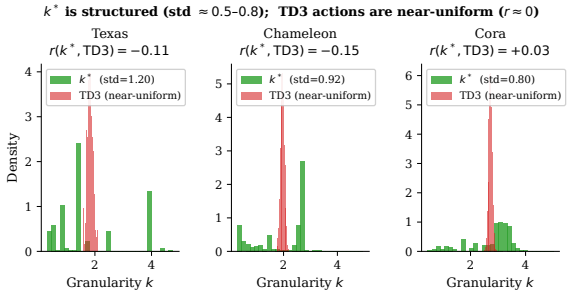
Dataset	Leakage-free	Oracle (leaky)	Δ
Texas	73.0	67.6	$+5.4$
Wisconsin	75.8	77.1	-1.3
Actor	33.1	32.9	$+0.3$

Table 4: One-dimensional hyperparameter sensitivity (accuracy range across 5 values, 3 seeds).

Parameter	Texas range	Wisc. range	Default in range?
α (α_h)	4.5%	3.3%	Yes
β (β_d)	1.8%	1.3%	Yes
γ (γ_{snr})	3.6%	2.6%	Yes



(a) Correlation between k^* and local homophily h_v.



(b) Correlation between TD3 actions and k^*.

Figure 2: Correlation analysis of node-level depths. (a) Our calculated analytic depth k^* correlates strongly with local homophily ($r \geq 0.92$). (b) The actions of the learned TD3 RL policy show near-zero correlation ($r \approx 0$) with k^*, indicating a lack of structured node-specific adaptation.

Table 5: Graph properties and kNN feature–label alignment (a_G). Bold values indicate datasets where the adaptive gate suppresses the implicit branch ($a_G < 0.35$).

Dataset	Homophily	kNN Align (a_G)	Suppressed
Texas	0.54	0.63	No
Wisconsin	0.57	0.69	No
Cora	0.87	0.55	No
Chameleon	0.76	**0.28**	**Yes**
Squirrel	0.70	**0.23**	**Yes**

protocol used for $\hat{h}_v$.

Leakage Audit

A potential concern is whether estimating local homophily from training labels introduces information leakage into the depth surrogate. We therefore compare our default *leakage-free* pipeline (pseudo-homophily from a classifier trained only on V_{train}) against an *oracle* variant that uses ground-truth labels on all nodes. Table 3 reports test accuracy over three seeds.

The mean per-run accuracy gap is only 1.5%, with no consistent advantage for the oracle variant. On Actor, the two settings are virtually identical ($\Delta = +0.3\%$). Pseudo-homophily also tracks oracle homophily well (mean Pearson $r = 0.75$, MAE $= 0.13$). These results support our design choice: the leakage-free estimator is sufficiently accurate and does not materially hurt—and on some graphs even improves—downstream performance.

The surprising Texas result ($\Delta = +5.4\%$ in favour of pseudo-homophily over oracle) has a structural explanation: Texas is a small graph (183 nodes) with strongly heterophilic structure, and oracle homophily pushes $k^*(v)$ to very shallow aggregation ($\approx k_{\min}$) for many nodes whose 1-hop neighbors are almost all from different classes. The pseudo-classifier, trained only on 60% of nodes and generalizing via feature similarity, produces smoother, less extreme estimates that avoid this degenerate all-$k_{\min}$ collapse. In effect, pseudo-homophily acts as a beneficial regularizer on small, noisy heterophilic graphs.

Hyperparameter Sensitivity

We sweep each exponent in Eq. (4) independently on Texas and Wisconsin (3 seeds each), holding the other two parameters at their defaults $(\alpha, \beta, \gamma) = (1.5, 0.35, 0.25)$. Table 4 summarizes the accuracy range per parameter.

Across both datasets, varying any single exponent changes accuracy by at most 4.5%, and the default settings lie within 1–2% of the best observed value in each 1D sweep. This indicates that performance is not brittle to the choice of (α, β, γ); the defaults used throughout our main benchmark are reasonable and near-optimal.

Ablations

Table 6 reports ablations of the three main design choices on Texas and Wisconsin (3 seeds, mean accuracy).

Table 6: Ablation study on design choices (mean test accuracy %).

Variant	Texas	Wisconsin
Full AGT-Implicit	73.0	75.8
w/o calibration Δk_v	72.1	74.9
w/o SNR input to β_v	72.4	75.1
w/o degree input to β_v	72.8	75.5
w/o implicit branch	71.2	73.6
w/o adaptive suppressor	70.5	74.3

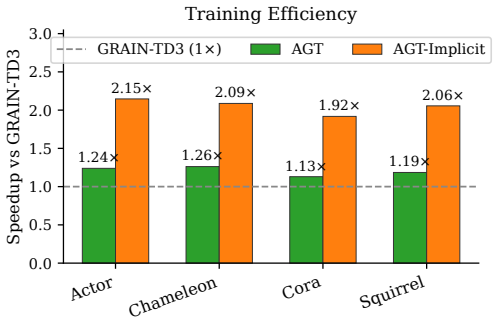


Figure 3: Training speedup of **AGT-Implicit** compared to GRAIN-TD3.

Each component contributes positively. Crucially, the *purely analytic* variant (w/o calibration) achieves $72.1\%/74.9\%$ on Texas/Wisconsin, still outperforming GRAIN-TD3 ($67.6\%/69.8\%$) by over 4.5%, confirming that the analytic depth formula drives the gain. Removing $\widetilde{\mathrm{SNR}}_v$ from gate inputs drops accuracy by 0.6%; removing the implicit branch costs 1.8% on Texas.

Computational Efficiency

Figure 3 shows the training speedup of **AGT-Implicit** vs. GRAIN-TD3. Table 2 reports total wall-clock time on Texas, giving the $30\times$ figure: GRAIN-TD3 spends $\sim$ 12 s/epoch on RL updates vs. our ~ 0.4 s. Figure 3 reports GNN-phase-only speedups (1.9–$2.1\times$) averaged across all nine datasets, reflecting the reduced RL warmup cost.

Conclusion and Limitations

We showed that per-node adaptive aggregation depth can be accurately determined via a theory-motivated closed-form surrogate derived under a degree-corrected CSBM. Our architecture, **AGT-Implicit**, is computed in a strictly leakage-free manner and replaces expensive RL controllers with this analytic formula. It matches or exceeds state-of-the-art RL controllers on seven of nine benchmarks while reducing training time by $30\times$ and controller parameters by $135\times$.

Limitations. (1) Our CSBM derivation uses a local-tree, two-class approximation; multi-class and high-degree-heterogeneity graphs remain future work. (2) The hard alignment threshold τ could be replaced with a differentiable estimator. (3) Head-to-head comparisons with concurrent RL-free methods (AD-GNN (Anonymous 2025a), BNA (Anonymous 2026)) under low-label regimes ($\leq 10\%$) are planned. (4) Combining k^* with a differentiable hop selector (e.g., NOL-GAT (Anonymous 2025b)) or dynamic topology rewiring (e.g., DRTR (Anonymous 2024)) could further improve performance.

References

Abu-El-Haija, S.; Perozzi, B.; Kapoor, A.; Alipourfard, N.; Lerman, K.; Harutyunyan, H.; Steeg, G. V.; and Galstyan, A. 2019. MixHop: Higher-Order Graph Convolutional Architectures via Sparsified Neighborhood Mixing. In *ICML*.

Anonymous. 2024. DRTR: Dynamic Topology Rewiring for Long-Range Graph Signals. *arXiv preprint arXiv:2406.17281*.

Anonymous. 2025a. AD-GNN: Node-Specific Aggregation Depth via Theoretical Analysis. *arXiv preprint arXiv:2511.06608*.

Anonymous. 2025b. NOL-GAT: Non-Local Graph Attention with Per-Node Hop Selection. *arXiv preprint arXiv:2502.06927*.

Anonymous. 2026. Bayesian Neighborhood Aggregation for Graph Neural Networks. *arXiv preprint arXiv:2602.05358*.

Baranwal, A.; Fountoulakis, K.; and Jagannath, A. 2022. Graph Convolution for Semi-Supervised Classification: Improved Linear Separability and Out-of-Distribution Generalization. In *ICML*.

Baranwal, A.; Fountoulakis, K.; and Jagannath, A. 2023. Effects of Graph Convolutions in Multi-layer Networks. In *ICLR*.

Bo, D.; Wang, X.; Shi, C.; and Shen, H. 2021. Beyond Low-frequency Information in Graph Convolutional Networks. In *AAAI*.

Chen, M.; Wei, Z.; Huang, Z.; Ding, B.; and Li, Y. 2020. Simple and Deep Graph Convolutional Networks. In *ICML*.

Chien, E.; Peng, J.; Li, P.; and Milenkovic, O. 2021. Adaptive Universal Generalized PageRank Graph Neural Network. In *ICLR*.

Defferrard, M.; Bresson, X.; and Vandergheynst, P. 2016. Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering. In *NeurIPS*.

He, M.; Wei, Z.; Huang, Z.; and Xu, H. 2021. BernNet: Learning Arbitrary Graph Spectral Filters via Bernstein Approximation. In *NeurIPS*.

Kipf, T. N.; and Welling, M. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *ICLR*.

Klicpera, J.; Bojchevski, A.; and Günnemann, S. 2019. Predict then Propagate: Graph Neural Networks meet Personalized PageRank. In *ICLR*.

Klicpera, J.; Weißenberger, S.; and Günnemann, S. 2019. Diffusion Improves Graph Learning. In *NeurIPS*.

Li, X.; Zhu, R.; Cheng, Y.; Shan, C.; Luo, S.; Li, D.; and Qian, W. 2022. Finding Global Homophily in Graph Neural Networks When Meeting Heterophily. In *ICML*.

Lim, D.; Hohne, F.; Li, X.; Huang, S. L.; Gupta, V.; Bhalerao, O.; and Lim, S.-N. 2021. Large Scale Learning on Non-Homophilous Graphs: New Benchmarks and Strong Simple Methods. In *NeurIPS*.

Luan, S.; Hua, C.; Lu, Q.; Zhu, J.; Zhao, M.; Zhang, S.; Chang, X.-W.; and Precup, D. 2022. Revisiting Heterophily for Graph Neural Networks. In *NeurIPS*.

Pei, H.; Wei, B.; Chang, K. C.-C.; Lei, Y.; and Yang, B. 2020. Geom-GCN: Geometric Graph Convolutional Networks. In *ICLR*.

Song, Y.; Zhou, C.; Wang, X.; and Lin, Z.-M. 2023. Ordered GNN: Ordering Message Passing to Deal with Heterophily and Over-smoothing. In *ICLR*.

Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. In *ICLR*.

Xu, K.; Hu, W.; Leskovec, J.; and Jegelka, S. 2019. How Powerful are Graph Neural Networks? In *ICLR*.

Xu, S.; Yin, J.; Li, D.; and Gao, F. 2025. GRAIN: Multi-Granular and Implicit Information Aggregation Graph Neural Network for Heterophilous Graphs. In *AAAI*.
Zhu, J.; Yan, Y.; Zhao, L.; Heimann, M.; Akoglu, L.; and Koutra, D. 2020. Beyond Homophily in Graph Neural Networks: Current Limitations and Effective Designs. In *NeurIPS*.